
A Unifying Perspective on Language Model Representations: From Filler-Role Structure to Mechanistic Interpretability

Zhang Enyan

Department of Computer Science
Yale University
New Haven, CT
enyan.zhang@yale.edu

R. Thomas McCoy

Department of Linguistics & Wu Tsai Institute
Yale University
New Haven, CT
tom.mccoy@yale.edu

Abstract

A wide range of methods have been proposed for interpreting language models, delivering important insights into their inner workings. However, different methods and their resulting insights stand in relative isolation: what could the underlying structure of language models be, such that they give rise to all our interpretations? In this work, we propose using Tensor Product Representations (TPRs; [62]) as a unifying hypothesis. TPRs give a concrete proposal for how compositional structure could be represented in vector space — as filler-role bindings. We show, both mathematically and empirically, that TPRs can unify several prior interpretability methods: additive analogies, linear probing, sparse autoencoders, and activation patching. Mathematically, we show that these methods can all be derived from TPRs. Empirically, we apply the derivations to a range of different models — from small toy models to LLMs — to construct instances of each of the above interpretability methods; these constructed variants perform comparably to their standard variants. We view this work as a step toward what interpretability will ideally provide: a unified account of the nature of neural networks, corroborated not just by individual observations but also by an explanation of the connections between them.¹

1 Introduction

In a classic story, several people are sent into a completely dark room containing an elephant. None of them have encountered an elephant before. One feels the elephant’s trunk and concludes that elephants are like snakes. Another feels a leg and concludes that elephants are like trees. Another feels a tusk and concludes that elephants are like spears, and the others make observations based on an ear, a side, and a tail (which are like a fan, a wall, and a rope, respectively). Taken together, these observations leave a puzzle: What sort of entity could the elephant be? What underlying structure could explain all of the properties — the snake-like properties, the tree-like properties, the spear-like properties, etc.? The answer, of course, is a structure like this: 

A similar situation is found when trying to understand neural network representations: interpretability work has made many important discoveries about neural network representations, but it is not obvious what structure underlies these observations. For instance, these representations support additive analogies [42, 52]; they can be linearly decoded [12, 1, 65, 24]; they can be decomposed into linear combinations of concepts [28, 57, 16]; and they can be edited (e.g., via activation patching) in ways that modify behavior [18, 70, 41, 21, 33]. All of these findings provide windows into the underlying

¹Code is available at <https://github.com/Rock-Z/Unifying-Interp-TPRs>

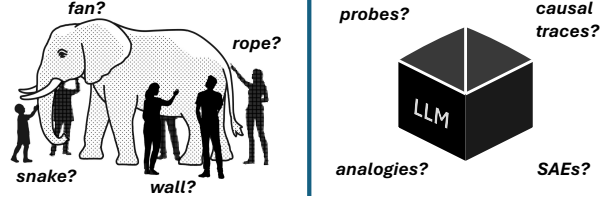


Figure 1: **Left:** Partial observations of an elephant point to a variety of properties—e.g., the trunk is snake-like and the ears are fan-like. All these partial observations can together be explained by the elephant’s overall structure. **Right:** Similarly, different language model interpretability methods expose consistent but partial descriptions of the same underlying model. Here we ask what the model’s underlying structure could be such that it supports all of these observations.

structure of the representations being analyzed. What might the nature of this structure be, such that it can explain all of the properties — the analogical properties, the decodability properties, etc. — that have been observed? An answer to this question would move the field toward a more complete picture of neural networks by unifying insights from different methods, behaviors, and types of models (Figure 1).

We posit that an account of a network’s representations should ideally be both precise and explanatory:

- (1) **Precise:** The account should provide an interpretable closed-form equation capturing the structure of the representations, such that representations can be constructed independently of the network being analyzed.
- (2) **Explanatory:** The account should provide explanations of how representations can be analyzed and edited in ways that are compatible with empirical observations.

In this paper, we propose using Tensor Product Representations (TPRs; [62]), a formalism for encoding information in vector space, as a unifying account of neural network representations. Building on this formalism, we make the following contributions:

1. We show that the closed-form equation provided by TPRs can be used to approximate the hidden states of a variety of neural networks across different tasks, corroborating prior work [39, 63, 40] and extending to new types of networks.
2. Mathematically, we derive several existing interpretability methods from TPR operations, unifying them under the same structural assumption.
3. Empirically, we apply the mathematical derivations to construct additive analogies, linear probes, sparse autoencoders, and activation patching from a single TPR approximation, finding performance that is comparable to applying these techniques in the standard ways.

The first contribution shows that the TPR formalism is precise (desideratum (1)), and the second and third contributions show that it is explanatory (desideratum (2)). This work shows that a formal theory of representational structure based on TPRs can connect multiple interpretability results, moving the field toward a more unified understanding of the structure of neural representations.

2 Background and related work

This paper brings together two areas of literature concerning the structure of vector representations: mechanistic interpretability and vector symbolic architectures. Here we introduce these lines of literature, which are further discussed in Section 8.

2.1 Mechanistic interpretability and the Linear Representation Hypothesis

Interpretability research aims to reverse engineer the representations and algorithms used by neural networks [2, 5, 47, 44], and it uses a wide range of methods. The most prominent existing proposal for unifying multiple methods is the Linear Representation Hypothesis (e.g., [50]), which posits that

information is encoded linearly inside neural networks. The Linear Representation Hypothesis is consistent with, and often assumed by, much interpretability work. Though it is often invoked informally or implicitly, this hypothesis can also be formalized. Below we give a possible formalization:

(3) The Linear Representation Hypothesis

Concept level: $\{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_n\}$

Vector level: $c_i = \text{emb}(\mathcal{C}_i)$, $h = \sum_{i=1}^n c_i$

That is, at an abstract, conceptual level, a neural network’s hidden state h is viewed as encoding a set of concepts $\mathcal{C}_1$ through $\mathcal{C}_n$. To translate abstract concepts to the vector representation h , each concept $\mathcal{C}_i$ is encoded with a vector embedding $\text{emb}(\mathcal{C}_i)$ and these embeddings are summed to obtain h .

2.2 Vector symbolic architectures and Tensor Product Representations

While the Linear Representation Hypothesis has supported many successful analyses, its expressivity is limited in ways that seem at odds with capturing the full structure of neural network representations. For example, one can train a small network to reverse sequences (e.g., $2, 1, 7 \mapsto 7, 1, 2$; Section 3.1). A natural way to encode $2, 1, 7$ under the linear representation hypothesis is as a set of three concepts: 2, 1, and 7. However, this “bag-of-numbers” fails to distinguish $2, 1, 7$ from reorderings such as $2, 7, 1$. A possible fix would be to treat each *position-content* pair as an atomic concept, but this produces a large number of concepts and fails to capture how concepts are related. For larger-scale networks, the issue is even more pronounced: e.g., for natural language, creating an atomic concept for each possible combination of a syntactic slot and a word that could fill that slot would require a number of concepts that seems hopelessly large, yet modern LLMs excel at processing language.

It seems, then, that an alternative account that can represent position-content combinations systematically rather than as atomic units is needed. Fortunately, decades of past research in Vector Symbolic Architectures and related formalisms have produced many approaches that share the same goal [62, 54, 53, 17, 56, 15, 31, 32]. Here, we focus on one specific approach in this tradition: Tensor Product Representations [61].

A Tensor Product Representation (TPR) encodes structure by binding a filler (content element) and a role (structural position), where each possible filler or role is represented as a vector embedding. E.g., the sequence $2, 1, 7$ can be thought of as the set of filler-role pairs “ $(2, 1^{\text{st}})$, $(1, 2^{\text{nd}})$, $(7, 3^{\text{rd}})$ ”. To encode filler-role pairs in a TPR, we first define the vocabulary of filler and role embedding vectors, $F = \{f_0, f_1, f_2, \dots\} \in \mathbb{R}^{d_f}$ and $R = \{r_{1^{\text{st}}}, r_{2^{\text{nd}}}, r_{3^{\text{rd}}}, \dots\} \in \mathbb{R}^{d_r}$. Then, each filler-role pair is composed together using the tensor product² as the binding operation, and all filler-role pairs are aggregated with summation. The TPR representing $2, 1, 7$ is thus $f_2 \otimes r_{1^{\text{st}}} + f_1 \otimes r_{2^{\text{nd}}} + f_7 \otimes r_{3^{\text{rd}}}$. Formally, the encoding for a set of filler-role pairs $\{f_i, r_i\}_{i=1}^n$ is

$$\text{encode}(\{f_i, r_i\}_{i=1}^n) = \sum_{i=1}^n f_i \otimes r_i \in \mathbb{R}^{d_f \times d_r}. \quad (1)$$

A TPR preserves the information it encodes: given vectors $\{u_1, u_2, \dots, u_n\}$ such that $r_i \cdot u_j = \delta_{ij}$, where δ is the Kronecker delta, right-multiplying u_j with a TPR recovers the filler bound to role j in a process known as unbinding:³

$$\sum_{i=1}^n f_i \otimes r_i u_j = \sum_{i=1}^n f_i r_i^\top u_j = \sum_{i=1}^n f_i \delta_{ij} = f_j. \quad (2)$$

More generally, TPRs can be constructed for a wide range of content types by using appropriate types of fillers and roles. For example, natural language can be encoded using words as fillers and syntactic positions (e.g., *subject*) as roles. Moreover, TPR binding and unbinding are simple linear or bilinear operations, making them straightforward to implement.

2.3 The Tensor Product Representation hypothesis

The structural expressivity of TPRs motivated [39] and [40] to hypothesize that neural network representations are structured as TPRs. In this paper, we adopt this hypothesis and term it the Tensor Product Representation hypothesis:

²As we use vectors as filler/role embeddings and no recursive bindings, this is equivalent to vector outer products.

³Such unbinding vectors are guaranteed to exist as long as the role vectors are linearly independent; see Appendix B.

(4) The Tensor Product Representation Hypothesis

Concept level: $\{(\mathcal{R}_1, \mathcal{F}_1), \dots, (\mathcal{R}_n, \mathcal{F}_n)\}$

Vector level: $r_i = \text{emb}(\mathcal{R}_i)$, $f_i = \text{emb}(\mathcal{F}_i)$, $h = \sum_{i=1}^n f_i \otimes r_i$

In the next sections, we first provide empirical evidence that TPRs can approximate neural network representations. We then show that this fact can be used to derive and explain a range of interpretability methods.

3 Approximating neural network representations with TPRs

Equation 3 shows the forward pass of a Tensor Product Encoder (TPE; [39]), which operationalizes TPRs into an architecture for approximating neural networks. A TPE takes in a set of filler-role pairs and outputs a linearly transformed TPR:

$$\text{TPE}(\{f_i, r_i\}_{i=1}^n) = W \text{vec}(\text{encode}(\{f_i, r_i\}_{i=1}^n)) + b. \quad (3)$$

Figure 2 presents an example of training a TPE to approximate the encoder hidden state of an RNN: Target activations are obtained from the frozen RNN; then, the TPE receives the input encoded as filler-role pairs;⁴ it is then trained to minimize the mean squared error loss with the target activation.

In interpretability research about representations, there are often two central questions: (i) what concepts are encoded in the model, and (ii) what representational structure/geometry is used to encode those concepts? Our focus is on (ii), such that our experiments would benefit from having little uncertainty about (i)—that is, if we are reasonably confident about what content is present, then we can run experiments that isolate the structural question (ii) because the answer to (i) is already reasonably clear and does not require, e.g., complex unsupervised feature discovery. Further, if we know which concepts to expect, we can use that information as the ground truth in probe evaluations, making evaluation easier than what is possible in settings without a clear ground truth. Accordingly, we apply the TPE architecture to two domains that each involve a fixed structure: digit sequences with a fixed length of 6 and English sentences with a fixed sentence structure of subject-verb-object (both discussed in the rest of this section). These fixed structures make it straightforward to hypothesize which concepts/features are likely to be encoded. Future work could extend our analyses to domains with more variable structure.

3.1 Synthetic sequences

Dataset: We first consider simple algorithmic tasks that involve copying and reversing randomly generated sequences. We generate 50,000 sequences of length 6 with a vocabulary of 20 for each task, split into train/validation/test following an 80:10:10 ratio. Details can be found in Appendix F.1.

Models: We train 1-layer encoder-decoder models: recurrent networks (RNNs; [10]), gated recurrent units (GRUs; [6]), and long short-term memory networks (LSTMs; [25]) with embedding size 64 and hidden size 256. This setup results in six different conditions, one for each architecture-task combination. All trained networks reach perfect accuracy; Appendix F.2 details training and results.

TPE Approximation: We freeze encoder-decoder models and train Tensor Product Encoders to reconstruct their final encoder hidden state. The TPEs receive, as input, sequences encoded as role-filler pairs of the position (1 through 6) and identity (1 through 20) of each token. The training process and results are detailed in Appendix F.3. Averaged across all six conditions, TPEs achieve an average substitution accuracy of 99.97% and R^2 of 0.9385; R^2 quantifies how well the approximation captures the variation in activation values, and substitution accuracy—the proportion of the time that the decoder produces the correct output when given the approximation rather than the true encoding—quantifies how well the approximation captures the structure that is behaviorally relevant. The

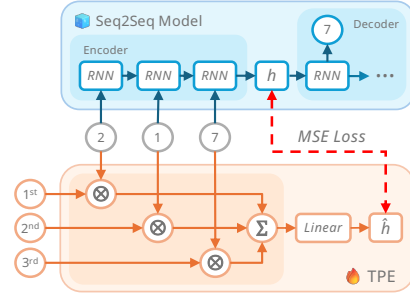


Figure 2: A Tensor Product Encoder training step, where it is trained to approximate representations in a frozen neural network.

⁴We manually specify the filler-role structure of an input in this work, though it can be made unsupervised ([63]).

strong approximation metrics are evidence that the encodings being analyzed are indeed structured as linearly-transformed TPRs.

3.2 SVO sentences

Dataset: Does TPE approximation extend beyond toy tasks to the representation of natural language in state-of-the-art transformers? In order to answer this question, we create a set of structured sentences with the template the <subject> will <verb> the <object>. The subject and object are each sampled from a list of 77 occupations (e.g., “linguist”, “chef”), and the verb is sampled from 5 possible verbs. See Appendix G.1 for more details.

Models: We analyze a set of language models and embedding models across various sizes. Qwen3-8B [72], OLMo2-13B [48], and GPT-OSS-20B [49] (language models), and ModernBERT [71], EmbeddingGemma [69] and Qwen3-Embedding-8B [73] (embedding models). We compute embeddings and representations for these models following their standard usage guidelines. For language models, we analyze the representation of the period token, which [58] and [40] find captures the content of the preceding sentence.

TPE Approximation: We train Tensor Product Encoders to approximate the representations for each model. To construct the input for the TPEs, each sentence is translated into three filler-role pairs for the subject, verb, and object of the sentence. For example, “the actor will see the chef.” is translated into the pairs (actor, subject), (see, verb), (chef, object). Appendix G.2 details this set of experiments; the TPEs achieve an R^2 over 0.90 for all embedding models and an R^2 over 0.60 for all LLMs, indicating that they capture most of the variance in these models. The R^2 is more modest for the LLMs than for the embedding models, perhaps because the LLM vectors that we analyzed (i.e., the period representations) have not been explicitly optimized to act as sentence representations.

4 Additive analogies

Past work has found additive analogy structures within word embeddings; e.g., [42] observed analogy quartets like “king – queen $\approx$ man – woman” where a relationship between two embeddings is represented as a fixed vector difference. Here we first show that representations in our settings also obey such analogies. We then show that additive analogies can be explained, both mathematically and empirically, as an effect of TPR structure.

4.1 Performing analogy experiments for our setups

Synthetic sequences: We construct analogy quartets A, B, C, D for the sequence tasks by substituting tokens from a sequence, such as the following: $\langle 2 \ 1 \ 7 \rangle - \langle 3 \ 1 \ 0 \rangle + \langle 3 \ 5 \ 0 \rangle \approx \langle 2 \ 5 \ 7 \rangle$. We sample quartets by substituting up to three positions from an initial sequence (see Appendix H.1) and report top- k accuracy by ranking the cosine similarity between the analogy vector (left-hand side) and embeddings of all sequences that appeared in an evaluation quartet, and checking if the target (right-hand side) is among the top- k .⁵ We find that the embeddings of trained models are highly structured, with a top-1 accuracy of 99.2% (Figure 3).

SVO sentences: We generate quartets A, B, C, D by finding A and D within the test set such that they differ by either subject or object, and then find B, C that shares the same difference (see Appendix H.2 for details). We apply the same evaluation procedure to sentence-embedding models and language-model hidden states, comparing the analogy vector against all sentences (Figure 3).

4.2 TPRs explain additive analogies

Additive analogies assume that substituting a component in a representation corresponds to a constant vector offset. Here, we show that this is exactly what the TPR hypothesis predicts. As an example, assume the encoder representation is (approximately) a TPE:

$$\langle x \rangle = W \operatorname{vec} \left(\sum_s f_{x,s} r_s^\top \right) + b,$$

⁵An important difference our evaluation and that of [42] is that we do not discard the embedding of the input sequence, and we use all embeddings as the target comparison set. We make this change to address the concerns raised by [34]. See Appendix H.3 for the full definition and implementation.

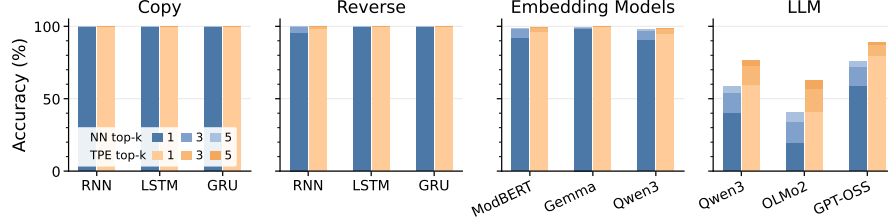


Figure 3: Additive analogy accuracy (%) evaluated with neural network representations (NN) and TPE-constructed analogy vectors (TPE). Analogies use encoder hidden states for recurrent networks, sentence embeddings for embedding models, and last token hidden state for LLMs. TPE analogies are constructed via Equation 4. TPE analogies match or exceed the accuracy of NN analogies.

where $f_{x,s}$ is the filler embedding at role/position s in input x . Consider two sequences where the difference is changing only the 2nd position from 1 to 5:

$$A = \langle 2 \ 1 \ 7 \rangle, \quad D = \langle 2 \ 5 \ 7 \rangle.$$

Substituting in the TPE approximation, we would predict the difference between the two representations to be

$$\langle 2 \ 5 \ 7 \rangle - \langle 2 \ 1 \ 7 \rangle = W \text{vec}(f_5) r_{2\text{nd}}^\top - W \text{vec}(f_1) r_{2\text{nd}}^\top = W \text{vec}(f_5 - f_1) r_{2\text{nd}}^\top,$$

as all remaining filler-role pairs and the bias terms cancel. For any other pair (B, C) that differs by the same substitution (e.g., $B = \langle 3 \ 1 \ 0 \rangle$ and $C = \langle 3 \ 5 \ 0 \rangle$), the same cancellation applies. Rearranging terms then yields the standard four-vector analogy ($\langle 2 \ 1 \ 7 \rangle - \langle 3 \ 1 \ 0 \rangle + \langle 3 \ 5 \ 0 \rangle = \langle 2 \ 5 \ 7 \rangle$). Generalizing, if we let S be the set of roles where two TPRs B and C differ, there is

$$\langle C \rangle - \langle B \rangle = W \text{vec} \left(\sum_{s \in S} (f_{C,s} - f_{B,s}) r_s^\top \right), \quad (4)$$

which predicts that additive analogies hold because they construct filler-role pair differences.

4.3 Additive analogies can be constructed from TPEs

Using the trained TPE for each model, we compute the analogy offset directly with Equation 4, without requiring the embeddings for B and C , and subtract it from the embedding of A . We evaluate with the same ranking-based procedure as Section 4.1. Across sequences and SVO sentences (Figure 3), TPE-constructed analogies match or outperform standard vector analogies despite requiring fewer forward passes (one instead of three) to compute the left-hand side of the analogy.

Figure 4 shows that offsets between ModernBERT sentence embeddings are similar across analogous sentence pairs (cosine similarity 0.89 ± 0.03). TPE binding differences match the offsets closely (0.94 ± 0.01), consistent with the account that the offsets represent filler-role binding differences.

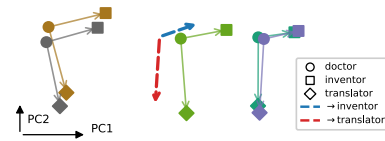


Figure 4: PCA projection of ModernBERT encodings for sentence triplets that differ only in subject. TPE-predicted embedding differences, shown in dashed arrows, closely align with those of sentence triplets.

5 Linear probing

Linear probes are classifiers (i.e., $\text{logits} = Wh + b$) that predict features from representations. They are widely used to discover implicitly encoded information within neural network representations [12, 1, 24, 65, 37]. In this section, we first train linear probes on our task setups. Then, we show that linear probes can be explained by and constructed from TPRs.

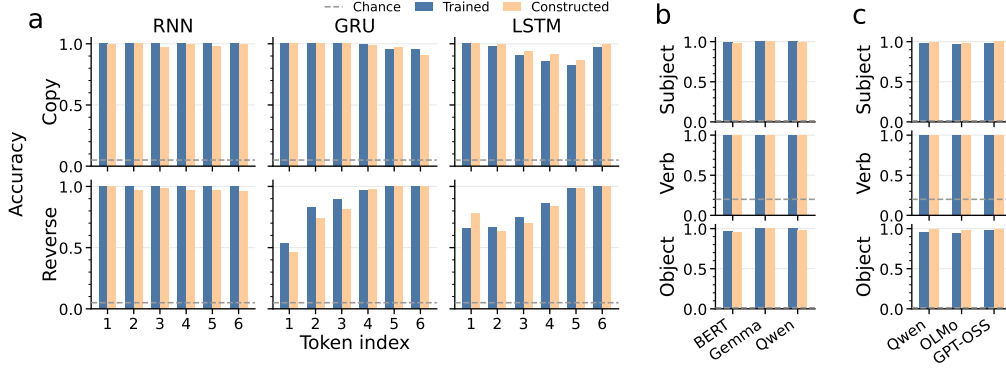


Figure 5: Accuracy comparison between trained linear probes and probes constructed from TPEs. (a) Probes that classify the identity of a token at a specific position (1 through 6) in the sequence; (b) Probes that classify the subject, verb, and object of a sentence given sentence embeddings; (c) Probes that classify subject, verb, and object given a LLM’s period token representation at the last layer.

5.1 Performing probing experiments for our setups

For the synthetic-sequence models, we train one probe per position to take in the encoder’s final hidden state and predict the number at the i -th position (e.g., $2\ 1\ 7\ 5\ 10\ 5 \mapsto 1$ for the “second” probe, Appendix F.4). Probes overall perform well, with near-perfect accuracies on RNNs and lower accuracies on a portion of positions of the GRUs/LSTMs. For the SVO sentence setting, we train three probes for each model to predict the subject, verb, and object from an input representation (Appendix G.3). Trained probes achieve high accuracies across all models and syntactic positions.

5.2 TPRs explain linear probing

A probe is a linear map from hidden states to semantic labels, the same as TPR unbinding (Eq. 2). Under the TPR hypothesis, probe parameters can therefore be viewed as performing an unbinding mapping, which we can construct explicitly from a trained TPE via three steps: (i) invert the TPE’s output layer to recover an approximate TPR encoding from the neural network representation, (ii) unbind from the recovered TPR to extract the filler, and (iii) project the recovered filler embedding onto the filler vocabulary to obtain classification logits. Full derivations are in Appendix D. As all of the steps are linear operations, they can be composed together into the following linear probe weights:

$$W_{\text{probe}} = F(u_j^\top \otimes I_{d_f})W^+, \quad b_{\text{probe}} = -W_{\text{probe}} b. \quad (5)$$

5.3 Linear probes can be constructed from TPEs

We utilize the above derivations to construct multiple probes from a single TPE and compare them to trained probes. Constructed probes largely match trained probe accuracy across both digit sequences and SVO sentences (Figure 5), achieving near-perfect accuracies at positions where the trained probes perform well, and similarly worse accuracies at the positions where trained probes perform worse.

Beyond performance comparisons, the constructed probes also lend insight into the relationship between different probes (e.g., subject and object probes). Specifically, the success of the probe construction process (which reuses filler embeddings across positional probes) suggests that different linear probes might be drawing upon shared structural components in the inputs that they decode from — shared filler representations — despite differing in which role is unbound.

6 Sparse autoencoders

A sparse autoencoder (SAE) learns sparsely activating features that linearly reconstruct input activations. SAEs often yield semantically meaningful LLM features [28, 3, 16, 57, 64], though their efficacy remains debated (e.g., [51, 66, 35]). In this section, we show that SAEs can be mathematically and empirically explained by, and derived from, TPRs.

6.1 Performing SAE experiments for our setups

We train SAEs on the sentence representations: sentence embeddings for embedding models and period-token final hidden states for language models. To avoid having to heuristically assign interpretations to features while still enabling tests of how well model representations can be decomposed into SAE features, we follow [35] and [30] in defining a set of ground-truth concepts against which SAE features can be compared. The ground-truth concepts that we use are pairings of a syntactic position and a word (e.g., subject-actor, verb-see).

We train two types of SAEs: top- k SAEs, and supervised SAEs with an auxiliary loss aligning features to ground-truth concepts (Appendix I). We report reconstruction scores (R^2) and *feature quality*, which measures for each concept the probability the feature activates more on examples where the concept is present than on ones where it is absent. Both SAE types perform well in reconstruction (Table 1), and supervised SAEs also achieve high feature quality. Top- k SAEs achieve lower feature quality, but this is expected since they are not supervised in what the features should be.

6.2 SAEs can be derived from TPRs

A canonical SAE derives feature values from activations with a linear layer followed by a nonlinear activation function, and then reconstructs the input with these features. The first layer, which maps from activations to concept space, can be derived from the TPR unbinding operation in a similar way as was done for linear probes. The weights and biases for an SAE can be constructed from a TPE as follows, where $W_{enc,i}$ and $b_{enc,i}$ give the SAE feature for filler $f_\star$ bound to role r_j (full derivations are in Appendix E):

$$W_{enc,i} = W^{+\top} u_j \otimes f_\star, \quad b_{enc,i} = -W_{enc,i}^\top b, \quad (6)$$

6.3 SAEs can be constructed from TPEs

We construct SAEs by instantiating one feature per filler–role pair, applying Equation 6 and concatenating the resulting features into W_{enc}, b_{enc} , and set W_{dec} as the pseudoinverse of W_{enc} . Table 1 shows the results of TPE-constructed SAEs compared to trained ones for both embedding-model and language-model representations. Constructed SAEs also reconstruct well (average $R^2 = 0.96$), though their R^2 is lower than that of their trained counterparts; a potential explanation is that TPEs are constrained to represent a filler–role pair as either present or absent, while SAEs are more expressive in that they give each feature a continuous weight. Overall, the comparison between trained and constructed SAEs shows that the fact that model encodings can be decomposed via SAEs can be explained via the TPR hypothesis.

Table 1: SAE performance.

Model	SAE type	R^2	Quality (%)
ModBERT	Top- k	0.9803	72.52
	Supervised	0.9676	99.99
	TPE-constr.	0.9289	99.77
Gemma	Top- k	0.9889	92.63
	Supervised	0.9866	97.26
	TPE-constr.	0.9679	99.98
Qwen3-Emb	Top- k	0.9785	80.84
	Supervised	0.9648	99.40
	TPE-constr.	0.9163	99.53
Qwen3-8B	Top- k	0.9710	94.11
	Supervised	0.9979	99.78
	TPE-constr.	0.9932	94.92
OLMo2	Top- k	0.9843	93.97
	Supervised	0.9894	99.85
	TPE-constr.	0.9822	99.32
GPT-OSS	Top- k	0.9897	97.94
	Supervised	0.9909	99.45
	TPE-constr.	0.9846	99.92

7 Activation patching

The previous sections make connections between the TPR hypothesis and a range of interpretability methods used for analyzing representations. A natural question to ask, then, is whether model behavior is causally affected by the TPR-like structure that is at the heart of our analyses. In this section, we hypothesize that the edits made by causal intervention methods such as activation patching [70] implicitly work by modifying TPR filler–role binding structure. We show that TPEs can be used to construct activation patching interventions with outcomes comparable to standard activation patching, providing evidence that the TPR structure we have identified in the previous sections is causally implicated in model behavior.

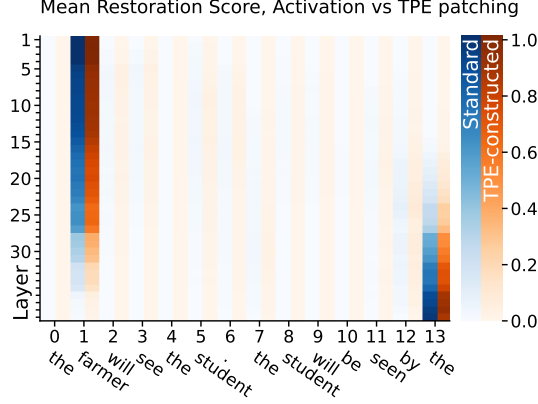


Figure 6: Comparison of restoration scores between standard activation patching (blue hues) and TPE-constructed patching (orange hues), on OLMo-2-13B, across layers and 14 token positions in the prompt. The two methods produce nearly identical results.

7.1 Performing patching experiments for our setups

We start by considering an activation patching task based on our SVO-structured sentences; the expected behavior is that the network will fill in the blank with the subject of the first sentence (e.g., *linguist* in the *source* example below):

Source: the linguist will see the student. the student will be seen by the _____

Destination: the actor will see the student. the student will be seen by the _____

An activation patching experiment intervenes on the forward pass of the *destination* prompt by replacing the activation at a specific layer and token with the analogous activation from the *source* prompt. If the activation is responsible for the model’s output, then the model’s response would also change to the response it would produce given the source prompt. We run an exploratory experiment on an autoregressive language model, OLMo-2-13B [48]. We run token-level activation patching on the same set of three autoregressive language models as seen in Sections 5 and 6, using active/passive SVO prompts. For each trial, we cache all activations from the source prompt, then patch a single layer and token position in the destination prompt. To quantify the effect of a patching trial, we compute *restoration score*, which measures how much the model’s behavior is restored via patching.⁶ Consistent with prior work [70], we find that the effective patch position shifts from the subject to the final token from early to late layers.

7.2 TPRs explain activation patching

A patching experiment can only be effective if the activation being patched is systematically structured in a way such that replacing an activation does not collapse the model’s performance but instead modifies its behavior accordingly. If the patched representations are TPRs that systematically encode the content of the sentence, then patching would indeed be successful. Further, the source/destination difference would be in the filler bound to the subject role, such that the patched activation should relate to the original activation via a binding difference:

$$h_{dest.} = h_{src.} - W((f_{src,subj} - f_{dest,subj}) \otimes r_{subj}) \quad (7)$$

This would predict, in turn, that activation patching does not require two separate forward passes to compute the source and target activations, as $h_{dest.}$ can instead be constructed by subtracting the binding difference from $h_{src.}$, seen in Equation 7.

7.3 Activation patching can be constructed via TPEs

We train TPEs, one for each layer of the chosen language models, to approximate its activations with the same filler-role structure as the SVO sentences task. All TPEs are trained with filler dimension 128 and role dimension 4, 100 epochs, learning rate 0.002 with cosine scheduling, and batch size 256.

⁶Here, we follow best practices by Heimersheim and Nanda [22] and use NNSight [13] to run patching experiments.

Then, we re-run activation patching on all token positions across all layers, constructing $\hat{h}_{dest.}$ with TPEs following Equation 7. Crucially, note that the TPE predicts an *activation difference* between the source and destination prompts, and computes the destination prompt representation using that difference instead of another language model forward pass. Figure 6 compares the mean restoration score between the two sets of patching experiments across all layers of OLMo and all token positions.

To quantify the similarity of standard activation patching and TPE-constructed patching, we additionally compute the Mean Absolute Error (MAE) and the Pearson Correlation Coefficient (r) between the two sets of patching restoration scores. The results are reported below:

- OLMo-2-13B: $r = 0.9998$, $MAE = 0.00098$
- Qwen3-8B: $r = 0.99999$, $MAE = 0.00052$
- GPT-OSS-20B: $r = 0.99991$, $MAE = 0.00135$

As both metrics and Figure 6 demonstrates, TPE-constructed patching achieves near-identical results as standard activation patching, correctly reproducing both the magnitude of the effect and the shift of information position, suggesting that TPR structure can serve as an explanation for the mechanisms underlying patching.

7.4 Relationship to other intervention-based methods

In the above experiment, we explicitly make comparisons between TPE-constructed patching and a two-example patching setup. More broadly, our comparison can also be connected to the assumptions of intervention-based methods that hypothesize that linearly adding and subtracting to representations can causally affect model behaviors in meaningful ways: for example, steering vectors [67, 23, 55] and causal abstraction [18, 19]. This assumption is consistent with the TPR hypothesis, and we connect this assumption to activation patching.

8 Discussion

We have demonstrated that a range of mechanistic interpretability methods can be explained with a single underlying structure, Tensor Product Representations. In doing so, we also deliver new insights on how these methods operate: for additive analogies, we give a structural hypothesis for the vector differences used in analogies. For linear probing, we show that different probes can be seen as the unbinding of different roles to recover the same set of fillers. For sparse autoencoders, we show that a dictionary of features can contain further internal structure. For activation patching, we show that the difference between patched activations can be explained as a filler-role binding difference.

Towards structural insights in interpretability: A typical interpretability project usually aims to provide an explanation for some behavior of interest within a model. While the explanation can be seen as the main result, it also tests the underlying structural hypotheses. For much of the work that involves interpreting transformers, the hypothesis usually involves some information — e.g. syntax [24, 27], semantic objects [70, 41], or truth values [37, 26] — being represented in some specific way — e.g., linearly [1, 29, 4, 68], in activations [20, 28], or by an attention head [7, 70, 60].

These structural assumptions are often treated as technical details or simplifications, necessary for the resource and architectural constraints we operate under and justified by prior empirical results. In contrast, we argue that the success of experiments based on these structural assumptions is an important piece of evidence in its own right. In the short term, emphasizing structural insights helps connect results across models and tasks. If successful explanations of models share similar structural properties, then all can be seen as corroborating the same structural hypothesis. Starting with a structural hypothesis then allows for deriving different interpretability methods as downstream results supported by the same structure; e.g., the same TPE approximation can be used to derive multiple analyses.

As a longer term goal, it also seems that interpretability should provide broad, structural descriptions that persist across individual behaviors and architectures. Here we take inspiration from early biology and cognitive science. Biology grew from its early descriptive stages of individual species and phenomena to finding unifying regularities across domains [38, 9]. Cognitive science emerged from

the unification of analyses from different disciplines [46, 43]. In a similar spirit, we envision that a future “science of language models” should also seek to deliver structural insights that answer questions concerning the nature of neural networks with respect to a broad class of models.

Relationship to the Linear Representation Hypothesis: The Linear Representation Hypothesis can also be taken as a unifying theory for mechanistic interpretations [50]. TPRs encapsulate this hypothesis but also add greater structure. Thus, TPRs can serve as a more expressive structural hypothesis that can address two notable limitations of the Linear Representation Hypothesis.

First, filler and role bindings extend beyond the “bag of concepts” view to explain the structure used to combine concepts, offering a theory of how position-content information can be compositionally stored in a single activation that the Linear Representation Hypothesis cannot explain. This type of expressivity is not only powerful but also considered essential for a range of cognitively significant capabilities [45, 14], many of which are now important targets of explanation in language model interpretability. As a concrete demonstration of such expressivity, we perform an exploratory experiment training TPEs on datasets where a portion of the filler-role pairs are held out from the training set, and we show that the approximation generalizes to these held-out examples (Appendix J).

The Linear Representation Hypothesis is also at odds with a growing body of empirical work showing that neural network concepts are not always encoded in a one-dimensionally linear manner [8, 11]. TPRs are multi-dimensionally linear and can capture structures such as the one [8] describes: indeed, our GRUs and LSTMs in Section 5 represent sequences in ways linear probes cannot fully recover, yet the TPEs trained to approximate them can achieve perfect substitution accuracy.

9 Conclusion

We have shown that the Tensor Product Representation formalism can unify four popular interpretability methods. Expanding such unification to additional methods and/or tasks can potentially deliver additional insights into how interpretability findings relate to each other and to the literature on vector symbolic architectures. Another avenue of work would be towards explaining how TPRs can account for complex, varying structures that go beyond the structural expressivity of the Linear Representation Hypothesis (e.g., syntax trees). More broadly, building theories of the shared structure that underlies the full spectrum of analysis techniques is an important goal for interpretability research—a goal that the current paper makes progress toward.

Acknowledgments

We would like to thank Yumeng Yan for the creating the illustration in Figure 1. For helpful discussion, we are grateful to Najoung Kim, Sophie Hao, Arman Cohan, Robert Frank, Alex Lew, the Computational Linguistics at Yale lab, and the Yale Mechanistic Interpretability group. Any errors are our own.

References

- [1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2017. URL <https://openreview.net/forum?id=ryF7rTqgl>.
- [2] David Bau, Jun-Yan Zhu, Hendrik Strobelt, Agata Lapedriza, Bolei Zhou, and Antonio Torralba. Understanding the role of individual units in a deep neural network. *Proceedings of the National Academy of Sciences*, 117(48):30071–30078, 2020. doi: 10.1073/pnas.1907375117. URL <https://www.pnas.org/doi/abs/10.1073/pnas.1907375117>.
- [3] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.

- [4] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=ETKGuby0hcs>.
- [5] Nick Cammarata, Shan Carter, Gabriel Goh, Chris Olah, Michael Petrov, Ludwig Schubert, Chelsea Voss, Ben Egan, and Swee Kiat Lim. Thread: Circuits. *Distill*, 2020. doi: 10.23915/distill.00024. <https://distill.pub/2020/circuits>.
- [6] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1179. URL <https://aclanthology.org/D14-1179/>.
- [7] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT’s attention. In Tal Linzen, Grzegorz Chrupała, Yonatan Belinkov, and Dieuwke Hupkes, editors, *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286, Florence, Italy, August 2019. Association for Computational Linguistics. doi: 10.18653/v1/W19-4828. URL <https://aclanthology.org/W19-4828/>.
- [8] Róbert Csordás, Christopher Potts, Christopher D Manning, and Atticus Geiger. Recurrent neural networks learn to store and generate sequences using non-linear representations. In Yonatan Belinkov, Najoung Kim, Jaap Jumelet, Hosein Mohebbi, Aaron Mueller, and Hanjie Chen, editors, *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 248–262, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.blackboxnlp-1.17. URL <https://aclanthology.org/2024.blackboxnlp-1.17/>.
- [9] Charles Darwin. *On the origin of species: A facsimile of the first edition*. Harvard University Press, 1964.
- [10] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. ISSN 0364-0213. doi: [https://doi.org/10.1016/0364-0213\(90\)90002-E](https://doi.org/10.1016/0364-0213(90)90002-E). URL <https://www.sciencedirect.com/science/article/pii/036402139090002E>.
- [11] Joshua Engels, Eric J Michaud, Isaac Liao, Wes Gurnee, and Max Tegmark. Not all language model features are one-dimensionally linear. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=d63a4AM4hb>.
- [12] Allyson Ettinger, Ahmed Elgohary, and Philip Resnik. Probing for semantic evidence of composition by means of simple classification tasks. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 134–139, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/W16-2524. URL <https://aclanthology.org/W16-2524/>.
- [13] Jaden Fried Fiotto-Kaufman, Alexander Russell Loftus, Eric Todd, Jannik Brinkmann, Koyena Pal, Dmitrii Troitskii, Michael Ripa, Adam Belfki, Can Rager, Caden Juang, Aaron Mueller, Samuel Marks, Arnab Sen Sharma, Francesca Lucchetti, Nikhil Prakash, Carla E. Brodley, Arjun Guha, Jonathan Bell, Byron C Wallace, and David Bau. NNSight and NDIF: Democratizing access to open-weight foundation model internals. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=MxbEiFRf39>.
- [14] Jerry A Fodor and Zenon W Pylyshyn. Connectionism and cognitive architecture: A critical analysis. *Cognition*, 28(1-2):3–71, 1988.
- [15] Stephen I Gallant and T Wendy Okaywe. Representing objects, relations, and sequences. *Neural computation*, 25(8):2038–2078, 2013.

- [16] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tcsZt9ZNKD>.
- [17] Ross W Gayler. Multiplicative binding, representation operators & analogy (workshop poster). 1998.
- [18] Atticus Geiger, Hanson Lu, Thomas F Icard, and Christopher Potts. Causal abstractions of neural networks. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, 2021. URL <https://openreview.net/forum?id=RmuXDtjDhG>.
- [19] Atticus Geiger, Zhengxuan Wu, Christopher Potts, Thomas Icard, and Noah Goodman. Finding alignments between interpretable causal variables and distributed neural representations. In Francesco Locatello and Vanessa Didelez, editors, *Proceedings of the Third Conference on Causal Learning and Reasoning*, volume 236 of *Proceedings of Machine Learning Research*, pages 160–187. PMLR, 01–03 Apr 2024. URL <https://proceedings.mlr.press/v236/geiger24a.html>.
- [20] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.446. URL <https://aclanthology.org/2021.emnlp-main.446/>.
- [21] Nicholas Goldowsky-Dill, Chris MacLeod, Lucas Sato, and Aryaman Arora. Localizing model behavior with path patching, 2023. URL <https://arxiv.org/abs/2304.05969>.
- [22] Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching, 2024. URL <https://arxiv.org/abs/2404.15255>.
- [23] Roee Hendel, Mor Geva, and Amir Globerson. In-context learning creates task vectors. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9318–9333, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.624. URL <https://aclanthology.org/2023.findings-emnlp.624/>.
- [24] John Hewitt and Christopher D. Manning. A structural probe for finding syntax in word representations. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4129–4138, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1419. URL <https://aclanthology.org/N19-1419/>.
- [25] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [26] Guan Zhe Hong, Nishanth Dikkala, Enming Luo, Cyrus Rashtchian, Xin Wang, and Rina Panigrahy. A implies B: Circuit analysis in LLMs for propositional logical reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=M0U8wUow8c>.
- [27] Phu Mon Htut, Jason Phang, Shikha Bordia, and Samuel R. Bowman. Do attention heads in BERT track syntactic dependencies?, 2019. URL <https://arxiv.org/abs/1911.12246>.
- [28] Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=F76bwRSLek>.

- [29] Dieuwke Hupkes, Sara Veldhoen, and Willem Zuidema. Visualisation and ‘diagnostic classifiers’ reveal how recurrent and recursive neural networks process hierarchical structure. *J. Artif. Int. Res.*, 61(1):907–926, January 2018. ISSN 1076-9757.
- [30] Adam Karvonen, Benjamin Wright, Can Rager, Rico Angell, Jannik Brinkmann, Logan Riggs Smith, Claudio Mayrink Verdun, David Bau, and Samuel Marks. Measuring progress in dictionary learning for language model interpretability with board game models. In *ICML 2024 Workshop on Mechanistic Interpretability*, 2024. URL <https://openreview.net/forum?id=qzsDKwGJyB>.
- [31] Denis Kleyko, Mike Davies, Edward Paxon Frady, Pentti Kanerva, Spencer J. Kent, Bruno A. Olshausen, Evgeny Osipov, Jan M. Rabaey, Dmitri A. Rachkovskij, Abbas Rahimi, and Friedrich T. Sommer. Vector symbolic architectures as a computing framework for emerging hardware. *Proceedings of the IEEE*, 110(10):1538–1571, October 2022. ISSN 1558-2256. doi: 10.1109/jproc.2022.3209104. URL <http://dx.doi.org/10.1109/JPROC.2022.3209104>.
- [32] Denis Kleyko, Dmitri A. Rachkovskij, Evgeny Osipov, and Abbas Rahimi. A survey on hyperdimensional computing aka vector symbolic architectures, part i: Models and data transformations. *ACM Computing Surveys*, 55(6):1–40, December 2022. ISSN 1557-7341. doi: 10.1145/3538531. URL <http://dx.doi.org/10.1145/3538531>.
- [33] János Kramár, Tom Lieberum, Rohin Shah, and Neel Nanda. Atp*: An efficient and scalable method for localizing LLM behaviour to components, 2024. URL <https://arxiv.org/abs/2403.00745>.
- [34] Tal Linzen. Issues in evaluating semantic spaces using word analogies. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 13–18, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/W16-2503. URL <https://aclanthology.org/W16-2503/>.
- [35] Aleksandar Makelov, Georg Lange, and Neel Nanda. Towards principled evaluations of sparse autoencoders for interpretability and control. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=1Nj173JKjB>.
- [36] Alireza Makhzani and Brendan Frey. k-sparse autoencoders, 2014. URL <https://arxiv.org/abs/1312.5663>.
- [37] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=aaJyHYjjjsk>.
- [38] Paolo Mazzarello. A unifying concept: the history of cell theory. *Nature cell biology*, 1(1): E13–E15, 1999.
- [39] R. Thomas McCoy, Tal Linzen, Ewan Dunbar, and Paul Smolensky. RNNs implicitly implement tensor-product representations. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=BJx0sjC5FX>.
- [40] R Thomas McCoy, Paul Soulos, Tal Linzen, and Paul Smolensky. The emergent symbolic structure of artificial neural networks. *arXiv*, 2026.
- [41] Kevin Meng, David Bau, Alex J Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=-h6WAS6eE4>.
- [42] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space, 2013. URL <https://arxiv.org/abs/1301.3781>.
- [43] George A Miller. The cognitive revolution: a historical perspective. *Trends in cognitive sciences*, 7(3):141–144, 2003.

- [44] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=9XFSbDPmdW>.
- [45] Allen Newell. Physical symbol systems. *Cognitive science*, 4(2):135–183, 1980.
- [46] Allen Newell and Herbert A Simon. Computer science as empirical inquiry: Symbols and search. In *ACM Turing award lectures*, page 1975. 2007.
- [47] Chris Olah. Mechanistic interpretability, variables, and the importance of interpretable bases. <https://www.transformer-circuits.pub/2022/mech-interp-essay>, June 2022. Transformer Circuits Thread.
- [48] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Allyson Ettinger, Michal Guerquin, David Heineman, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Jake Poznanski, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 OLMo 2 Furious, 2025. URL <https://arxiv.org/abs/2501.00656>.
- [49] OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrlyov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- [50] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Causal Representation Learning Workshop at NeurIPS 2023*, 2023. URL <https://openreview.net/forum?id=T0Po0Jg8cK>.
- [51] Gonalo Paulo and Nora Belrose. Sparse autoencoders trained on the same data learn different features. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=EjInprGpk9>.
- [52] Jeffrey Pennington, Richard Socher, and Christopher Manning. GloVe: Global vectors for word representation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1162. URL <https://aclanthology.org/D14-1162/>.
- [53] Tony A Plate. Holographic reduced representations. *IEEE Transactions on Neural networks*, 6(3):623–641, 1995.

- [54] Jordan B Pollack. Recursive distributed representations. *Artificial Intelligence*, 46(1-2):77–105, 1990.
- [55] Itamar Pres, Laura Ruis, Ekdeep Singh Lubana, and David Krueger. Towards reliable evaluation of behavior steering interventions in LLMs. In *MINT: Foundation Model Interventions*, 2024. URL <https://openreview.net/forum?id=7xJcX2gbm9>.
- [56] D.A. Rachkovskij. Representation and processing of structures with binary sparse distributed codes. *IEEE Transactions on Knowledge and Data Engineering*, 13(2):261–276, 2001. doi: 10.1109/69.917565.
- [57] Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, Janos Kramar, Rohin Shah, and Neel Nanda. Improving sparse decomposition of language model activations with gated sparse autoencoders. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=zLBlin2zvW>.
- [58] Anton Razzhigaev, Matvey Mikhalechuk, Temurbek Rahmatullaev, Elizaveta Goncharova, Polina Druzhinina, Ivan Oseledets, and Andrey Kuznetsov. LLM-microscope: Uncovering the hidden role of punctuation in context memory of transformers. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 7772–7779, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.432. URL <https://aclanthology.org/2025.findings-naacl.432/>.
- [59] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. URL <https://arxiv.org/abs/1908.10084>.
- [60] Aaditya K. Singh, Ted Moskovitz, Felix Hill, Stephanie C. Y. Chan, and Andrew M. Saxe. What needs to go right for an induction head? a mechanistic study of in-context learning circuits and their formation, 2024. URL <https://arxiv.org/abs/2404.07129>.
- [61] Paul Smolensky. Analysis of distributed representation of constituent structure in connectionist systems. In *Neural Information Processing Systems*, 1987.
- [62] Paul Smolensky. Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, 46(1):159–216, 1990. ISSN 0004-3702. doi: 10.1016/0004-3702(90)90007-M. URL <https://www.sciencedirect.com/science/article/pii/000437029090007M>.
- [63] Paul Soulos, R. Thomas McCoy, Tal Linzen, and Paul Smolensky. Discovering the compositional structure of vector representations with role learning networks. In *Proceedings of the Third BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, page 238–254. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.blackboxnlp-1.23. URL <http://dx.doi.org/10.18653/v1/2020.blackboxnlp-1.23>.
- [64] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- [65] Ian Tenney, Dipanjan Das, and Ellie Pavlick. BERT rediscovers the classical NLP pipeline. In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1452. URL <https://aclanthology.org/P19-1452/>.

- [66] Claire Tian, Katherine Tian, and Nathan Hu. Measuring sparse autoencoder feature sensitivity, 2025. URL <https://arxiv.org/abs/2509.23717>.
- [67] Eric Todd, Millicent Li, Arnab Sen Sharma, Aaron Mueller, Byron C Wallace, and David Bau. Function vectors in large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=AwxytyMwaG>.
- [68] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering, 2024. URL <https://arxiv.org/abs/2308.10248>.
- [69] Henrique Schechter Vera, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, Iftekhar Naim, Joe Zou, Feiyang Chen, Daniel Cer, Alice Lisak, Min Choi, Lucas Gonzalez, Omar Sanseviero, Glenn Cameron, Ian Ballantyne, Kat Black, Kaifeng Chen, Weiyi Wang, Zhe Li, Gus Martins, Jinhyuk Lee, Mark Sherwood, Juyeong Ji, Renjie Wu, Jingxiao Zheng, Jyotinder Singh, Abheesht Sharma, Divyashree Sreepathihalli, Aashi Jain, Adham Elarabawy, AJ Co, Andreas Doumanoglou, Babak Samari, Ben Hora, Brian Potetz, Dahun Kim, Enrique Alfonseca, Fedor Moiseev, Feng Han, Frank Palma Gomez, Gustavo Hernández Ábrego, Heseng Zhang, Hui Hui, Jay Han, Karan Gill, Ke Chen, Koert Chen, Madhuri Shanbhogue, Michael Boratko, Paul Suganthan, Sai Meher Karthik Duddu, Sandeep Mariserla, Setareh Ariafar, Shanfeng Zhang, Shijie Zhang, Simon Baumgartner, Sonam Goenka, Steve Qiu, Tanmaya Dabral, Trevor Walker, Vikram Rao, Waleed Khawaja, Wenlei Zhou, Xiaoqi Ren, Ye Xia, Yichang Chen, Yi-Ting Chen, Zhe Dong, Zhongli Ding, Francesco Visin, Gaël Liu, Jiageng Zhang, Kathleen Kenealy, Michelle Casbon, Ravin Kumar, Thomas Mesnard, Zach Gleicher, Cormac Brick, Olivier Lacombe, Adam Roberts, Qin Yin, Yunhsuan Sung, Raphael Hoffmann, Tris Warkentin, Armand Joulin, Tom Duerig, and Mojtaba Seyedhosseini. Embeddinggemma: Powerful and lightweight text representations, 2025. URL <https://arxiv.org/abs/2509.20354>.
- [70] Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=NpsVSN6o4u1>.
- [71] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Griffin Thomas Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2526–2547, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.127. URL <https://aclanthology.org/2025.acl-long.127/>.
- [72] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [73] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176>.

A Overview of appendices

These appendices provide mathematical derivations, experimental details, and extended results that complement the main text. The appendices start with tables listing notations and abbreviations used throughout the paper. Then, separate sections detail the synthetic sequence and SVO sentence datasets, then list model and training hyperparameters, followed by the evaluation procedures for linear probing, analogies, sparse autoencoders, and activation patching.

B Finding unbinding vectors

Given a set of role vectors $\{r_1, r_2, \dots, r_n\}$ and the role matrix R where each column is a role vector, *unbinding* vectors are vectors u_i such that for all u and r 's, $u_i \cdot r_j = \delta_{ij}$. Informally, the dot product between the unbinding vector and a role vector should be 1 if and only if they correspond, and should be 0 otherwise. Finding such unbinding vectors would allow for lossless information retrieval from TPRs, as Section 2.2 shows.

If we denote the unbinding matrix U , where each column is an unbinding vector corresponding to the role vector in the same column, it is not difficult to see that we have $U^\top R = I$ if and only if R is full-rank. As role vectors are real-valued, then as long as $|\{R\}| \leq d_r$, i.e., the number of roles is less than or equal to the dimension of role vectors, R is very likely full-rank.⁷ Then, finding the unbinding vectors simply requires computing the inverse of the role matrix.

For this reason, we limit our role dimensions choices to those larger than the number of roles in all of our experiments.⁸ We also compute the regularized pseudoinverse for the unbinding vectors as gradient descent finds noisy approximations of role vectors. We find that naively using the Tikhonov-regularized pseudoinverse with $\lambda = 0.1$ works well for all our experiments.

C Recovering a TPR from neural network hidden state

A shared component between constructing linear probes (Appendix D) and SAEs (Appendix E), or more generally, any application of TPR unbinding, is that they must first recover a TPR from the neural network hidden states.

This step is non trivial. To give an example, the RNN we trained TPEs to approximate (Section 3.1) has a hidden state size of 256, while the TPR binding has $64 \times 64 = 4096$ elements when flattened. As such, the projection weights W form a down projection for most of the Tensor Product Encoder setups, and recovering such a TPR would require computing an up-projection that entails an ill-conditioned pseudoinverse.

One option, of course, is to constrain the dimensions of fillers and role such that the TPR always has fewer elements than the activation it is trained to approximate. Empirically, however, we find that this is not necessary or helpful. Instead, we find that merely using the pseudoinverse of W (in an appropriately regularized form, as described below) is often possible, possibly because the flattened TPR does not span its full vector space. We offer no principled explanations for why this is the case, but provide a few observations from initial experiments:

1. If one trains a sequence-to-sequence model that reverses sequence with a hidden size of 400, and then trains TPEs with filler and role dimension of 20 to approximate it, computing the (pseudo)inverse of the output projection layer gives an inverse transform that is able to recover a TPR from its linearly transformed variant, but not the NN hidden states it is trained to approximate, even though the two are closely correlated and the linear projection layer is in principle invertible.
2. If one computes all of the singular values of the projection layer weight in the above-mentioned TPE, there will be a large number of small singular values that are approximately

⁷To make a hand-wavy argument, R will be full rank with probability 1 assuming any reasonable family of continuous distribution for r .

⁸TPRs also handle approximate unbinding as introduced in [62], which allows for packing significantly more role vectors within a fixed high-dimensional space under a certain unbinding error limit. Here we circumvent the need to consider approximate unbinding as it is outside the scope of the project.

0, suggesting that the projection layer is not in fact full rank. This finding is, in hindsight, unsurprising, as specifying a hyperparameter, the dimensionality of filler or role embeddings, bears no weight on the representational structure of the sequence-to-sequence network and the subspace it spans.

Both the observations above and the up-projection generally required are similar to finding the linear best fit of an ill-posed regression problem. We thus apply the solution, applying a regularization technique called the Tikhonov regularization:

$$W_{\lambda}^{+} = (W^{\top}W + \lambda I)^{-1}W^{\top}, \quad (8)$$

Where $\lambda \geq 0$. Intuitively, setting a larger λ “dampens” the effect of small singular values on the pseudoinverse. We find that setting this regularization parameter correctly is crucial to retrieving an approximate TPR from a neural network hidden state. In our empirical experiments, the output layer regularization λ is chosen with ternary search with respect to a batch of neural network activations and their filler-role encoding: we start with a range of $[10^{-12}, 10^{12}]$. At each iteration, we compute the pseudoinverse with the candidate regularization parameter, and then apply it on a batch of 128 neural network activations to obtain approximate TPRs and compute its MSE loss with the TPRs encoded with filler-role pairs. The algorithm repeats until it finds a $\pm 10\%$ range of possible regularization parameters. The pseudocode for the full algorithm is in Algorithm 1.

Algorithm 1 Ternary search for regularization using a batch objective

```
# inputs:
# batch_hidden: the neural network hidden states of a batch of data
# batch_filler_ids: the filler ids for the batch of data
# batch_role_ids: the role ids for the batch of data
# objective_fn: callable mapping log_lambda -> scalar loss
# log_lo, log_hi: search bounds in log10(lambda)
# tol: relative precision
# obtains:
# best_lambda: lambda minimizing objective_fn

def ternary_search_log_lambda(objective_fn, log_lo, log_hi, tol):
    precision = log10(1 + tol)

    while (log_hi - log_lo) > precision:
        left_third = (2 * log_lo + log_hi) / 3
        right_third = (log_lo + 2 * log_hi) / 3

        loss_left = objective_fn(left_third)
        loss_right = objective_fn(right_third)

        if loss_left < loss_right: # minimize
            log_hi = right_third
        else:
            log_lo = left_third

    best_log = 0.5 * (log_lo + log_hi)
    best_loss = objective_fn(best_log)
    return best_log, best_loss, (log_lo, log_hi)

# --- batch objective used for output-layer inversion ---
# inputs:
# batch_hidden: [B, D] hidden states to invert
# batch_filler_ids: [B, N], batch_role_ids: [B, N]
# model: provides output-layer inverse and ground-truth TPR

def batch_objective(log_lambda):
    W_inv = output_layer_inverse(model, l2_lambda=10**log_lambda)
    recovered_tpr = linear(batch_hidden, W_inv)
    gt_tpr = model_tpr(batch_filler_ids, batch_role_ids)
    return mse(recovered_tpr, gt_tpr)

# call:
best_log, best_loss, bracket = ternary_search_log_lambda(
    objective_fn=batch_objective,
    log_lo=LOG_LO,
    log_hi=LOG_HI,
    tol=TOL,
)
best_lambda = 10**best_log
```

D Constructing linear probes from TPRs

Outline The derivation takes three main steps:

1. recover $\text{vec}(E)$ from h .
2. unbind the recovered TPR with u_j
3. project the unbinding result, which is a predicted filler embedding, onto filler vocabulary

We provide a complete derivation of the analytic probe used in Section 5. We start with the standard TPR setup:

TPR setup Let fillers $f_i \in \mathbb{R}^{d_f}$ and roles $r_i \in \mathbb{R}^{d_r}$. The (unflattened/unprojected) TPR matrix is

$$E = \sum_{i=1}^n f_i r_i^\top \in \mathbb{R}^{d_f \times d_r}. \quad (9)$$

A Tensor Product Encoder maps E to a hidden state $h \in \mathbb{R}^{d_h}$ by

$$h = W \text{vec}(E) + b, \quad (10)$$

with projection $W \in \mathbb{R}^{d_h \times (d_f d_r)}$ and bias $b \in \mathbb{R}^{d_h}$. Let u_j be the unbinding vector for role r_j , so that $r_i^\top u_j \approx \delta_{ij}$.

Lemma 1 (unbinding identity). For any $E \in \mathbb{R}^{d_f \times d_r}$ and $u \in \mathbb{R}^{d_r}$,

$$E u = (u^\top \otimes I_{d_f}) \text{vec}(E). \quad (11)$$

Proof. This is the standard Kronecker/vec identity $(x^\top \otimes I_m) \text{vec}(A) = Ax$.

Lemma 2 (recovering the filler from h). Let W^+ denote the pseudoinverse of W , such that $W^+ W \approx I$. Then the filler f_j bound to role r_j can be (approximately) recovered via the following:

$$(u_j^\top \otimes I_{d_f}) W^+ (h - b).$$

Proof. First, the flattened TPR $\text{vec}(E)$ can be recovered via the following

$$h = W(\text{vec}(E)) + b \quad (12)$$

$$\text{vec}(\hat{E}) := W^+(h - b) = W^+((W(\text{vec}(E)) + b) - b) \approx \text{vec}(E) \quad (13)$$

Plugging $\text{vec}(\hat{E})$ into Equation 11 gives

$$\hat{E} u = (u_j^\top \otimes I_{d_f}) W^+ (h - b), \quad (14)$$

where the left hand side is the TPR unbinding operation (Equation 2). Thus,

$$\hat{f}_j = (u_j^\top \otimes I_{d_f}) W^+ (h - b), \quad (15)$$

which approximates the filler bound to role r_j . Note that when the inverse W^+ is the exact inverse ($W^+ W = I$), the filler is exactly recovered instead of approximately recovered.

Classification over the filler vocabulary Let $F \in \mathbb{R}^{|\mathcal{F}| \times d_f}$ be the matrix of fillers used for classification (rows are filler vectors, optionally normalized). The logits for role r_j are

$$\text{logits}(h) = F \hat{f}_j = F(u_j^\top \otimes I_{d_f}) W^+ (h - b). \quad (16)$$

Thus the analytic probe is linear in h with parameters

$$W_{\text{probe}} = F(u_j^\top \otimes I_{d_f}) W^+, \quad b_{\text{probe}} = -W_{\text{probe}} b. \quad (17)$$

This yields a fully specified linear probe with no additional training.

E Constructing SAEs from TPRs

Outline We consider deriving individual feature vectors, which we then compose into an SAE by constructing feature vectors for all possible concepts. The derivation takes 3 main steps, in a structure analogous to linear probe construction:

1. express a desired binding score as a linear function of $\text{vec}(E)$.
2. substitute the TPE inverse to obtain a linear detector in h .
3. embed the detector as an SAE feature via $(W_{\text{enc}}, b_{\text{enc}})$ and set a compatible decoder.

Setup. Let $h = W \text{vec}(E) + b$ be a linearly transformed TPR, with $E = \sum_i f_i r_i^\top$. An SAE with hidden features $z \in \mathbb{R}^m$ has the form

$$z = \text{ReLU}(W_{\text{enc}}h + b_{\text{enc}}), \quad (18)$$

$$\hat{h} = W_{\text{dec}}z + b_{\text{dec}}. \quad (19)$$

We construct features that “detect” concepts, i.e., $s(h) = d^\top h + c$.

Lemma 3 (binding detector) Let u_j be the unbinding vector for role r_j and let $f_\star$ be a filler embedding. Define

$$d_{(f_\star, r_j)} = (W^+)^{\top}(u_j \otimes f_\star), \quad c_{(f_\star, r_j)} = -d_{(f_\star, r_j)}^\top b. \quad (20)$$

Then the scalar

$$s_{(f_\star, r_j)}(h) = d_{(f_\star, r_j)}^\top h + c_{(f_\star, r_j)} \quad (21)$$

approximates the binding score $f_\star^\top E u_j$.

Proof. Using the TPE-constructed linear probe weights (Equation 16), and setting the set of filler embeddings F as a single filler $f_\star$,

$$f_\star^\top E u_j = (u_j \otimes f_\star)^\top W^+(h - b) = d_{(f_\star, r_j)}^\top h + c_{(f_\star, r_j)}. \quad (22)$$

Embedding into SAE parameters For a feature with detector (d, c) , set the k -th encoder row to $W_{\text{enc}}[k, :] = d^\top$ and bias $b_{\text{enc}}[k] = c$. Then the SAE feature is $\text{ReLU}(d^\top h + c)$, which gives the binding score $f_i^\top E u_j$ for the filler i -role j pair. This allows the SAE to reconstruct by setting $W_{\text{dec}} = W_{\text{enc}}^+$ and setting decoder bias such that it cancels the encoder bias, i.e. $b_{\text{dec}} = -W_{\text{enc}} b_{\text{enc}}$. The construction yields explicit SAE features for a collection of specific filler-role bindings derived from the TPE projection and unbinding vectors.

F The synthetic sequences task

F.1 Task and dataset

The sequence manipulation task is a sequence-to-sequence task that involves manipulating randomly generated sequences of tokens. We describe the complete data schema and generation procedure below.

We specify a vocabulary of 20 tokens. For clarity, we identify tokens with their index, $\{0, 1, 2, \dots, 18, 19\}$. Each unit of the sequence is chosen independently from a uniform distribution over the vocabulary. Then, depending on the task, the target output is generated by algorithmically manipulating the input: for the copy task, it is the same as the input sequence; for reverse, the positions of tokens are reversed. We give the two tasks and an example input–output pair of each task below in Table 2. A total of 50,000 examples, all of length 6, are generated for each task. They are split into 80% (40,000) train, 10% (5,000) evaluation, and 10% (5,000) test. The generated sequences are cached and preserved across training runs.

F.2 Sequence-to-sequence models

We train one-layer encoder–decoder RNNs, GRUs, and LSTMs on the two sequence tasks (copy, reverse) using the dataset described in Appendix F.1. The trained model checkpoints for each task/architecture are then reused across all sequence experiments (TPE approximation, linear probes, and analogies).

Table 2: Example input–output pairs for the sequence manipulation tasks: *copy* returns the input unchanged, and *reverse* outputs the reversed sequence.

Task	Input						Output					
Copy	2	1	7	5	10	5	2	1	7	5	10	5
Reverse	2	1	7	5	10	5	5	10	5	7	1	2

Special tokens To format the task in a way that is friendly to encoder–decoder models, we prepend and append special tokens to the sequences. For the encoders, each input sequence is prepended with `<bos>` and appended with `<sep>`. For example, the input sequence 2 1 7 5 10 5 translates to 8 input tokens to the encoder, `<bos> 2 1 7 5 10 5 <sep>`. The decoder starts generation with `<bos>` as input, and we take the decoder’s generation to be complete when it reaches 10 tokens or outputs an `<eos>` token.

Hyperparameters The models are all one-layer, with an embedding size of 64 and hidden state size of 256. The LSTM also has a cell state size of 256. The models do not have dropout. They are trained with the AdamW optimizer, with a batch size of 128, for 60 epochs. The learning rate follows a linear warmup over the first 100 steps, followed by a cosine decay schedule, peaking at 0.002. The best checkpoint, selected via validation split accuracy, is stored.

Evaluation and results The trained models are then evaluated for their accuracy on the test split. We compute two metrics: the *token* level accuracy, which is teacher-forced by having the decoder receive the ground-truth input at each step, and computing the proportion of times the decoder outputs the correct token. The *sequence* level accuracy uses no teacher forcing, instead letting the decoder generate until it emits an `<eos>` token or reaches 10 tokens, and computes the proportion of times the generated sequence matches the ground truth as a whole. The *token* accuracy setting mirrors training more closely, while the *sequence* accuracy measures the ability of the trained network to actually perform its training task. Both are reported in Table 3, and we see that the trained networks all achieve perfect accuracy.

Table 3: Sequence-to-sequence model test accuracies with token- and sequence-level metrics.

Task	RNN		LSTM		GRU	
	Token	Sequence	Token	Sequence	Token	Sequence
Copy	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Reverse	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000

F.3 Tensor Product Encoders

For each sequence-to-sequence model, we train a TPE to reconstruct the encoder’s final hidden state. These TPEs are then stored and reused across the experiments: analogy evaluation (Appendix H) and probe construction (Appendix D).

The output dimension of the TPEs trained to approximate RNNs and GRUs is 256. For LSTMs, the TPE approximates the *concatenated* vector composed of the encoder’s hidden state and cell state, so its output dimension is 512. The filler embeddings for the TPEs are the same size as the sequence-to-sequence model’s vocabulary size. For roles, we do a simple left-to-right assignment of roles, including special tokens. After adding in a null token used for padding, this results in a total of 9 role embeddings.

Hyperparameters After doing an exploratory analysis optimizing for a large range of hyperparameters, we find that TPE training is quite insensitive to hyperparameters. We thus heuristically choose one set of hyperparameters proven to be successful approximating some of the sequence-to-sequence models, and reuse across all tasks and architectures. All the TPEs have filler and role dimensions 64, and are trained with batch size 64, learning rate 0.002 with a constant learning rate schedule, for 20 epochs. The checkpoint achieving the lowest evaluation loss during training is saved.

Table 4: Tensor Product Encoder substitution R^2 and token and sequence-level accuracies on the test split.

Task	RNN			LSTM			GRU		
	Token	Sequence	R^2	Token	Sequence	R^2	Token	Sequence	R^2
Copy	1.0000	1.0000	0.9853	1.0000	1.0000	0.9754	0.9998	0.9990	0.9343
Reverse	1.0000	1.0000	0.8325	1.0000	0.9998	0.9721	0.9999	0.9996	0.9311

Table 5: Probe accuracy (%) comparison between trained vs analytically constructed probes on the sequence manipulation task. The results provided here correspond to the data used in plotting figure 5.

Model	Position	Copy		Reverse	
		Trained	Constructed	Trained	Constructed
GRU	1	100.00	100.00	58.70	57.34
	2	100.00	100.00	85.84	76.68
	3	99.88	99.96	91.52	84.70
	4	99.12	99.08	98.40	97.32
	5	94.32	86.84	100.00	99.96
	6	95.00	90.18	100.00	100.00
LSTM	1	100.00	100.00	65.86	75.62
	2	97.78	95.28	70.58	63.48
	3	91.12	86.70	75.36	68.24
	4	86.74	88.12	87.12	82.68
	5	85.14	71.68	98.90	98.82
	6	96.20	98.62	100.00	100.00
RNN	1	100.00	99.28	100.00	98.66
	2	100.00	95.28	100.00	98.74
	3	99.78	93.22	100.00	98.60
	4	99.86	95.56	100.00	98.38
	5	100.00	99.50	100.00	98.70
	6	100.00	100.00	100.00	97.74

Evaluation and results As introduced in Section 3, we evaluate trained TPEs via two metrics: the R^2 between the sequence-to-sequence model’s hidden state and the TPE’s approximation, and the substitution accuracy, which measures the accuracy of the decoder model if its initial hidden state is generated by the TPE instead of the encoder model. The results are reported in Table 4, where we see that all the TPE approximations achieve high R^2 and near-perfect substitution accuracies across both tasks.

F.4 Linear probes

We train linear probes that take in the final encoder hidden state to predict the token identity at each input position. This results in 6 probes (one for each possible position) for each of the 6 sequence-to-sequence models.

The probes are all trained using the train split of the synthetic sequences dataset, with labels generated by algorithmically identifying the token that takes a specific position.

Hyperparameters All 6 (number of positions/probes per model) $\times$ 2 (number of tasks) $\times$ 3 (number of architectures) probes share the same hyperparameters: they are trained for 5 epochs, with a batch size of 256, constant learning rate of 5×10^{-3} .

Evaluation and results We evaluate the probes on the test split and report accuracy as the proportion of times the probe’s most likely label matches the ground truth. The accuracies for trained probes are presented as blue bars in Figure 5. The exact values can be found in Table 5.

Table 6: Example generated sentences.

the carpenter will see the magician.
the tutor will help the engineer.
the violinist will see the dancer.
the umpire will help the librarian.
the intern will visit the scientist.

Table 7: List of the 77 occupation nouns used as <subject> and <object> in the SVO template.

professor	student	president	judge	senator	secretary	doctor	lawyer
scientist	banker	tourist	artist	author	actor	athlete	teacher
engineer	accountant	architect	chef	photographer	farmer	ambassador	astronaut
astronomer	blacksmith	baker	barber	biologist	butler	chemist	composer
cartoonist	coach	captain	carpenter	dancer	director	drummer	detective
explorer	economist	editor	governor	gardener	illustrator	intern	inventor
journalist	linguist	manager	magician	mayor	miner	mathematician	musician
novelist	nurse	painter	philosopher	physicist	politician	programmer	pilot
poet	reporter	referee	sailor	spy	translator	treasurer	technician
tutor	umpire	violinist	writer	librarian			

G The SVO sentences task

G.1 Structured sentences

For transformer language models, we base our experiments on a set of simple subject-verb-object (SVO) sentences of the following structure:

the <subject> will <verb> the <object>.

To generate the dataset, we chose a list of 77 nouns of occupations (e.g. lawyer, student, farmer), and 5 verbs, “see”, “help”, “visit”, “teach”, “call”. We selected these verbs because an occupation term is semantically reasonable as the subject or direct object for any of them. The dataset is generated by exhaustive combination of the above, creating a total of $77^2 \times 5 = 29645$ examples. They are then divided into train, evaluation, and test splits at the ratio of 80%:10%:10%, similar to the examples in the sequence task, resulting in 23,717 train examples, 2,965 evaluation examples, and 2,966 test examples. Tensor Product Encoders are trained with embeddings of sentences on the train split, selected on the evaluation split, and final metrics are reported on the test split. Since the dataset contains no duplicate sentences, all test sentences are unseen. Table 6 shows a few sample sentences generated by this procedure. The full list of occupations can be seen in Table 7.

G.2 Tensor Product Encoders

We train TPEs to reconstruct sentence embeddings for three embedding models. The TPEs receive as input 3 filler-role pairs, corresponding to the subject, verb, and object of the sentence, respectively. As an example, the sentence *the carpenter will see the magician.* will be translated, algorithmically, into the TPE input pairs {(carpenter, subject), (see, verb), (magician, object)}. This filler-role scheme captures the possible variations between different sentences. The period and the words *the* and *will* are constant across all sentences in the dataset, so we hypothesize that they do not need to be included in the representation of each sentence because the bias term (of Equation 3) should be able capture the invariant offset that would result from including them.

Encoding We use the SentenceTransformers library [59] to access the embedding models and encode sentence representations. All models are loaded with `bf16` datatype. For ModernBERT, we use its embedding model variant `nomic-ai/modernbert-embed-base`. The model is trained for vector embedding search, and requires an embedded sentence to be either a query or document. To align with its expected usage, we prepend “query_document” to all sentences before encoding them. Qwen3-Embedding-8B and EmbeddingGemma encode sentences out-of-the-box, so we directly use them to encode the sentences with no modifications.

Table 8: SVO TPE metrics on the held-out test split.

Type	Model	R^2	MSE
Embedding Model	ModernBERT	0.9396	4.27×10^{-5}
	EmbeddingGemma	0.9155	1.57×10^{-5}
	Qwen3-Embedding-8B	0.9000	9.51×10^{-6}
LLM	Qwen3-8B	0.7052	3.50×10^{-2}
	OLMo-2-13B	0.6440	3.28×10^{-2}
	GPT-OSS-20B	0.7423	5.16×10^{-1}

Table 9: SVO probe accuracy (%) for trained vs analytically constructed probes on SVO sentences.

Model	Role	Trained	Constructed
ModernBERT	Subject	99.26	89.61
	Verb	100.00	98.01
	Object	96.49	91.53
EmbedGemma	Subject	99.83	100.00
	Verb	100.00	99.83
	Object	99.87	99.80
Qwen3-8B	Subject	99.93	99.70
	Verb	100.00	100.00
	Object	99.87	98.31

For the LLM experiments, we use decoder-only language models and cache the final-layer hidden state at the sentence-final period token via Huggingface Transformers. We use this punctuation-token representation because it occurs after the full SVO sentence has been processed.

Hyperparameters We train TPEs to approximate the sentence embeddings previously generated with the hyperparameters listed below.

1. **ModernBERT:** Filler dimension 128, role dimension 4; trained for 100 epochs with an initial learning rate of 0.002, following cosine decay schedule.
2. **Qwen3-Embedding-8B:** Filler dimension 128, role dimension 4; trained for 100 epochs with an initial learning rate of 0.002, following cosine decay schedule.
3. **EmbeddingGemma:** Filler dimension 128, role dimension 4; trained for 100 epochs with an initial learning rate of 0.005, following cosine decay schedule.

For the LLM punctuation-token TPEs, we use the following hyperparameters:

1. **Qwen3-8B:** Filler dimension 256, role dimension 4; trained for 100 epochs with an initial learning rate of 0.004, following a cosine decay schedule.
2. **OLMo-2-13B:** Filler dimension 256, role dimension 4; trained for 100 epochs with an initial learning rate of 0.004, following a cosine decay schedule.
3. **GPT-OSS-20B:** Filler dimension 256, role dimension 4; trained for 100 epochs with an initial learning rate of 0.004, following a cosine decay schedule.

We evaluate TPEs at the end of every epoch on the evaluation split of the dataset, and save when the metrics are better than the last saved checkpoint. For each model, we choose the checkpoint with best evaluation loss and report its performance on the test split. The final metrics for all TPEs are reported in Table 8

G.3 Linear probes

We train linear probes to classify the subject, verb, and object given a sentence embedding. Probes are trained with all 23,717 sentences in the train set, batch size 256, and a constant learning rate

of 0.005. We train for 20 epochs for ModernBERT and EmbeddingGemma, and 50 epochs for Qwen3-Embedding-8B, confirming that evaluation loss plateaus by inspecting training logs. Due to the simplicity of the task, we did not perform automated hyperparameter searches. Analytic probes are evaluated with batch size 256 on the held-out test split. Results are reported for each probe (subject/verb/object) on the test split.

Each role probe predicts a 77-way subject/object label or a 5-way verb label. All trained probes perform well, and all but the object probe trained on ModernBERT reach near-perfect accuracy. The results are visualized in Figure 5b, and also presented in Table 9.

H Additive analogy construction and evaluation

Below we describe the exact procedure for constructing and evaluating additive analogy quartets on our task setups.

H.1 Synthetic sequences

The analogy quartets are set up as follows. Denote the 4 sequences of an analogy quartet A, B, C, D . We draw sequence A from the test split of each task. Given $A = (a_1, \dots, a_6)$ (since all sequences in the dataset are length 6), we sample a set of *substituted roles* $S \subset \{1, \dots, 6\}$ uniformly at random with $1 \leq |S| \leq 3$. We call the remaining positions, $R = \{1, \dots, 6\} \setminus S$, the *retained roles*. We then construct sequences

$$B = A^{(R \leftarrow \tilde{a})}, C = A^{(S \leftarrow \hat{a}, R \leftarrow \tilde{a})}, D = A^{(S \leftarrow \hat{a})},$$

where $S^{(P \leftarrow a)}$ denotes the *replacement* operation, replacing the positions in P with random tokens from $\hat{a}$ and $\tilde{a}$, two randomly generated sequences that are different at each position from the original sequence A . Thus, B and C differ only on S , and D is A with exactly those positions replaced. The analogy quartets all contain special tokens, so the recurrent encoders can encode them as normal inputs. Example (5) demonstrates a quartet constructed from the algorithm above:

(5) Example (sequence analogy):

$A = \langle \text{bos} \rangle \ 2 \ 1 \ 7 \ 5 \ 10 \ 5 \ \langle \text{sep} \rangle$
 $B = \langle \text{bos} \rangle \ 9 \ 3 \ 7 \ 8 \ 10 \ 4 \ \langle \text{sep} \rangle$
 $C = \langle \text{bos} \rangle \ 9 \ 3 \ 0 \ 8 \ 10 \ 4 \ \langle \text{sep} \rangle$
 $D = \langle \text{bos} \rangle \ 2 \ 1 \ 0 \ 5 \ 10 \ 5 \ \langle \text{sep} \rangle$

We evaluate analogies by sampling 1000 analogies per model and task. We compute analogy vectors using both the standard three-embedding method and the TPE-constructed method, then evaluate top- k accuracy using the algorithm described in Appendix H.3.

H.2 SVO sentences

For SVO sentences, we construct analogies from the test split by varying exactly one role (subject, verb, or object) while holding the other two fixed. For *subject* analogies, we group sentences by fixed (verb, object) values; within each group, for every ordered pair of distinct subjects ($\text{val}_A, \text{val}_C$), we seek another group with a different (verb, object) pair that contains both subjects. We then form A and D from the first group and B and C from the second group, yielding a quartet A, B, C, D where only the subject role varies. Object analogies are constructed analogously by grouping on (subject, verb) and varying the object role. We stop at the first matching second group for each ($\text{val}_A, \text{val}_C$) pair.

Analogies are ranked against all possible SVO sentences, although the quadruples themselves are built only from the test split. This evaluation procedure yields exactly 44,468 analogies in total (22,488 subject analogies and 21,980 object analogies). Below we show one such example quartet:

(6) Example (subject analogy):

$A = \text{"the doctor will see the patient ."}$
 $B = \text{"the doctor will help the student ."}$
 $C = \text{"the lawyer will help the student ."}$
 $D = \text{"the lawyer will see the patient ."}$

H.3 Evaluation

Top- k accuracy Given an analogy vector v (either $e(A) - e(B) + e(C)$ or the TPE-constructed variant) and a candidate pool of embeddings $\{e_i\}_{i=1}^N$, we rank candidates by cosine similarity

$$s_i = \frac{v^\top e_i}{\|v\|_2 \|e_i\|_2}.$$

Let i^* be the index of the target sentence/sequence D . The rank of the target is

$$\text{rank}(v) = 1 + \sum_{i=1}^N \mathbb{1}\{s_i > s_{i^*}\},$$

and top- k accuracy is the fraction of analogies with $\text{rank}(v) \leq k$.

A parallelized implementation Computing cosine similarity between vectors and ranking them sequentially is inefficient and takes an unreasonable amount of time even for our relatively small evaluation set. We implement the same computation in a vectorized way. First, cosine similarities are computed in batch between all embeddings. Then, we index the target’s similarity as an entry from the resulting cosine similarity matrix, and do distributed comparison between the target cosine similarity and every cosine similarity with masking. We present implementation pseudocode below:

Algorithm 2 Computing top- k accuracy for analogies

```
# inputs:
# V: analogy vectors, shape [B, D]
# E: candidate embeddings, shape [N, D]
# target_idx: index of target D for each analogy, shape [B]
# k: specifies top-k accuracy
# obtains:
# topk: the top-k accuracy

E = E / (norm(E, axis=1, keepdims=True) + 1e-12) # [N, D]
V = V / (norm(V, axis=1, keepdims=True) + 1e-12) # [B, D]

S = E @ V.T # [N, B] Cosine similarity
matrix
target_sim = S[target_idx, arange(B)] # [B]
ranks = 1 + (S > target_sim).sum(axis=0) # [B]

topk = mean(ranks <= k)
```

I Training and evaluating sparse autoencoders

We train SAEs on SVO sentence representations using two variants: top- k SAEs, and supervised SAEs, where the first is a standard performant SAE architecture, and the second is a modified architecture specifically suiting our purposes. For embedding models, the representations are sentence embeddings. For language models, the representations are cached final-layer hidden states at the sentence-final period token, using the same punctuation-token representations described in Appendix G.2. All SAEs are trained on the train split of the relevant SVO dataset, then evaluated on the held-out test split.

Throughout this section, we term a row in the SAE’s weight encoder weight matrix a “feature”, and the value corresponding to it in the intermediate layer of the SAE the “feature activation”. We term the explanation assigned to the feature the “concept”, which is often a semantic content-syntactic role pair, such as “subject-linguist”.

I.1 Reconstruction R^2

We report reconstruction R^2 and feature quality. For reconstruction, given inputs x and reconstructions $\hat{x}$, we compute

$$R^2 = 1 - \frac{\sum \|x - \hat{x}\|_2^2}{\sum \|x - \bar{x}\|_2^2}, \quad (23)$$

where $\bar{x}$ is the mean representation in the dataset used for evaluation. R^2 captures how well the SAE’s reconstruction recovers the input representations with respect to the total variations within the evaluation dataset, where 0 stands for (roughly speaking) no correlation, and 1 stands for perfect reconstruction. This metric allows us to quantify the quality of SAE reconstructions across models that widely differ in sizes.

I.2 Feature quality

Motivation A feature is meaningful if its activations correlate well with whether a given concept is present within the example — intuitively, the feature should activate highly when the concept it tracks is present, and should not activate (as highly) when the concept is not. We cash out this notion of “well-rankedness” by asking, for a given feature and concept, the probability that a positive example (containing the concept) is ranked above a negative example (not containing the concept) by that feature’s activation.

Definition For each feature k , let $a_i^{(k)}$ denote its activation on example i . Fix a concept y_k for this feature and define the positive and negative sets

$$P_k = \{i : \text{example } i \text{ contains } y_k\}, \quad N_k = \{i : \text{example } i \text{ does not contain } y_k\}.$$

We define the quality score for the feature evaluated on this concept as the probability a randomly sampled pair of positive-negative samples are *well-ranked*, i.e.,

$$\text{Quality}(k) = \frac{1}{|P_k||N_k|} \sum_{i \in P_k} \sum_{j \in N_k} \left[\mathbb{I}\{a_i^{(k)} > a_j^{(k)}\} + \frac{1}{2} \mathbb{I}\{a_i^{(k)} = a_j^{(k)}\} \right].$$

This metric gives 1.0 for perfect separation, and 0.0 for perfectly reversed ranking. To compute the quality score for an SAE, we take the average across the quality score of all its features. The pseudocode illustrating how we compute feature quality of an SAE in the code implementation is presented below:

Algorithm 3 Computing feature quality

```
# inputs:
# A: activations, shape [num_examples, num_features]
# example_labels: concepts that are present in each example (e.g., subject-linguist)
# feature_labels: assigned concept for each feature
# obtains:
# scores: list of quality scores for every feature

scores = []
for k in range(num_features):
    y = feature_labels[k]
    pos = where(example_labels == y)
    neg = where(example_labels != y)
    if len(pos) == 0 or len(neg) == 0:
        continue
    a_pos = A[pos, k] # [n_pos]
    a_neg = A[neg, k] # [n_neg]
    score = mean( (a_pos[:,None] > a_neg[None,:]) +
                  0.5 * (a_pos[:,None] == a_neg[None,:]) )
    scores.append(score)
```

Assigning concept to unsupervised SAEs For supervised and constructed SAEs, each feature is explicitly trained to correspond to a pre-defined concept. However, top- k SAEs are unsupervised, and we do not know a priori what concept a feature corresponds to, if anything. We thus run the above evaluation process for each feature with every possible concept on the train set, and assigning the feature the concept for which it performs best on. We then report the quality score by evaluating it again on the test set, this time specifying the concept as the one we previously assigned.

I.3 Top- k SAEs

We train top- k SAEs [36], which can be thought of as a way to modify standard SAEs by replacing the activation function with keeping only the k -highest activated units.

As there is no guarantee that the k -highest activated units follow the same distribution between training and test stages, we follow the standard procedure [16] of storing a moving average of the numerical threshold for top- k activations during training, and use the numerical threshold instead at evaluation time.

Following standard practices [64], we also employ *dead neuron resampling*: with randomly initialized features, it may be the case that some feature directions never get activated by examples in the training set. We thus keep a running record of features that have not been activated during training (or since the last resampling step), and resample them by randomly initializing them again. During training, we set an n *resamples* hyperparameter, which then defines evenly spaced resampling steps throughout training. For example, training for 20 epochs with 4 resampling steps will result in resampling at the end of the 4th, 8th, 12th, and 16th epochs. We have found that increasing the number of times resampling is performed to be essential for the SAEs to discover high-quality features that align with human expectations (e.g., subject-doctor).

Hyperparameters Due to standard concerns about hyperparameter sensitivity during SAE training [51], we perform a random search over the span of hyperparameters: $k \in \{4, 8, 16, 32\}$, hidden dimension $\in \{128, 256, 512, 1024\}$, batch size $\in \{64, 128, 256\}$, learning rate $\in [10^{-5}, 5 \times 10^{-3}]$ (sampled with log-uniform distribution), and resample times $\in \{1, 2, 5\}$.

We train each model for 200 epochs,⁹ where the first 5% (10 epochs) applies linear learning rate warmup, and heuristically select the checkpoint (among all hyperparameter search checkpoints) that achieves the best sparsity-reconstruction tradeoff: that is, the trained SAE should have sparsely activated features with a sparsity that aligns with human expectations. Any SAE that is more sparse will have a worse loss than the chosen one, and any SAE with lower loss will not be as sparse. In the end, the chosen top- k SAE checkpoints have the following hyperparameters:

- ModernBERT: hidden dimension 1024, $k = 8$. Learning rate = 6.658×10^{-4} , batch size = 256.
- Qwen3-Embedding-8B: hidden dimension 1024, $k = 32$. Learning rate = 1.508×10^{-3} , batch size = 256.
- EmbeddingGemma: hidden dimension 1024, $k = 8$. Learning rate = 4.815×10^{-4} , batch size = 128.

For the language-model punctuation-token experiments, the top- k baselines are selected from 40 total sweeps run over hidden dimension $\{512, 1024, 2048, 4096\}$, $k \in \{8, 16, 32, 64, 128\}$, batch size $\{128, 256\}$, learning rate sampled log-uniformly from $[10^{-5}, 5 \times 10^{-3}]$, and resampling times $\{2, 5, 8\}$. The sweep uses Weights and Biases’ Bayesian backend. These models are also trained for 200 epochs. The selected checkpoints optimize feature quality on the validation split, with the following hyperparameters:

- Qwen3-8B: hidden dimension 4096, $k = 64$. Learning rate = 1.69×10^{-5} , batch size = 256, resample times = 5.
- OLMo-2-13B: hidden dimension 2048, $k = 128$. Learning rate = 3.72×10^{-3} , batch size = 128, resample times = 2.

⁹This is a significantly larger number of epochs compared to what is commonly seen for SAE training, and we find this to improve results significantly as our training dataset is quite small (approximately 25 thousand examples).

- GPT-OSS-20B: hidden dimension 1024, $k = 128$. Learning rate = 4.19×10^{-3} , batch size = 128, resample times = 5.

Supervised SAEs A supervised SAE pre-assigns a concept for each of its features at initialization time, and as such always has 159 features (77 (subjects) + 77 (objects) + 5 (verbs)). In addition to standard SAE losses, it adds an auxiliary “supervision loss”, which is a cross-entropy loss between the SAE’s pre-ReLU intermediate activations and the ground truth concepts that are present. Each sentence in our dataset contains 3 concepts (one each for subject, verb, and object), so the target distribution the intermediate activations are trained to approximate is one that assigns three labels $1/3$ probability, and 0 to the rest of the labels. Formally, given an input embedding $x \in \mathbb{R}^d$, the encoder computes pre-activations and activations

$$a = W_e x + b_e, \quad z = \text{ReLU}(a), \quad (24)$$

and the decoder reconstructs

$$\hat{x} = W_d z + b_d. \quad (25)$$

The standard SAE loss is the sum of reconstruction error and an L_1 sparsity penalty on the activations:

$$\mathcal{L}_{\text{SAE}} = \|\hat{x} - x\|_2^2 + \lambda \|z\|_1, \quad (26)$$

where λ is the sparsity weight. The supervision loss is applied to the encoder *pre-activations* a . For each example we build a multi-label target vector $y \in \{0, 1\}^F$ with three ones (subject, verb, object) and normalize it to a distribution $\tilde{y} = y / \sum_j y_j = y/3$. We then compute a cross-entropy loss against the softmax of a :

$$\mathcal{L}_{\text{sup}} = - \sum_{j=1}^F \tilde{y}_j \log \text{softmax}(a)_j. \quad (27)$$

The total objective is

$$\mathcal{L} = \mathcal{L}_{\text{SAE}} + \alpha \mathcal{L}_{\text{sup}}, \quad (28)$$

where α is the supervision weight.

As cross-entropy loss biases features towards negative infinity, we find that setting the supervision weight and no additional L1 regularization is sufficient for sparse activations. The embedding-model SAEs are trained with sparsity penalty 0.0, and supervision weight 0.05 (ModernBERT, EmbeddingGemma) or 0.02 (Qwen3-Embedding-8B). The language-model SAEs use sparsity penalty 0.0, supervision weight 0.02, and a dead-latent threshold of 10^{-6} . All supervised SAEs are trained for 40 epochs, learning rate 2×10^{-3} , warmup ratio 0.05, and batch size 128. For the language-model SAEs, we checked seeds $\{13, 42, 101\}$ and found stable reconstruction at the displayed precision: mean R^2 is 0.9979 for Qwen3-8B, 0.9894 for OLMo-2-13B, and 0.9907 for GPT-OSS-20B.

J TPR substitutions systematically generalize

TPRs have a compositional structure that is systematically constructed from filler and role embeddings. If this predicted structure matches with that of the neural network representations that we are analyzing, then we would expect TPR approximations to *generalize*: that is, even if we train the TPR approximation on only a subset of possible filler-role combinations, the learned TPR approximation should nevertheless generalize to novel filler-role combinations in the evaluation set. Below we test this hypothesis on both the synthetic sequence tasks and the SVO sentences task, showing that in both situations the TPE approximation generalizes to novel filler-role pairs.

J.1 Synthetic Sequences

For the synthetic sequences experiments, we construct fixed-length-6 copy and reverse datasets in which one randomly chosen number-position pair is held out at each position: (2, 1), (7, 2), (13, 3), (4, 4), (18, 5), and (10, 6). The train, validation, and test splits contain no examples with any of these held-out pairs, while the generalization split contains 5000 examples, each of which contains at least one held-out pair. Thus the generalization split tests whether a TPE trained without a particular filler-role binding can still reconstruct the encoder state for sequences containing that binding.

We reuse the same frozen sequence-to-sequence models as in Appendix F.1. The TPEs are trained with the same hyperparameters as Appendix F.3. As before, we evaluate the TPEs with reconstruction

R^2 and substitution accuracy, where substitution accuracy measures decoder accuracy when the encoder hidden state is replaced by the TPE approximation.

Table 10: Digits filler-role holdout results. Test examples contain no held-out filler-role pairs; generalization examples contain at least one held-out filler-role pair.

Task	Model	Test		Generalization	
		Substitution Acc.	R^2	Substitution Acc.	R^2
Copy	RNN	1.0000	0.9863	1.0000	0.9835
	LSTM	1.0000	0.9949	0.9634	0.9882
	GRU	0.9994	0.9600	0.8230	0.9005
Reverse	RNN	1.0000	0.8344	1.0000	0.8238
	LSTM	0.9994	0.9888	0.9464	0.9758
	GRU	0.9998	0.9497	0.9098	0.9000

As seen in Table J.1, the held-out substitution results remain strong. Averaged across the six TPEs, the generalization split has $R^2 = 0.9287$ and substitution accuracy 0.9404. The RNN TPEs show perfect accuracy on both the test and generalization splits. The GRU and LSTM TPEs show some performance drops, but still performs well ($\geq 90\%$) on the OOD generalization set.

J.2 SVO sentence embeddings

We construct an analogous holdout split for the SVO sentence embeddings. Because verbs occur only in the verb role in this dataset, we hold out noun-role pairs rather than verb-role pairs. Seven nouns are held out in subject position (professor, artist, ambassador, coach, gardener, musician, and referee) and seven different nouns are held out in object position (secretary, engineer, barber, drummer, linguist, physicist, and technician). The train, validation, and test splits contain no sentence with these noun-role pairs, while the generalization split contains all sentences with at least one held-out noun-role pair. This yields 19,600 training sentences, 2450 validation sentences, 2450 test sentences, and 5145 generalization sentences.

We train TPEs with the same filler-role encoding, embedding models, and hyperparameters as Appendix G.2. The evaluation again compares the TPE reconstruction to the frozen sentence embedding, but for this setting we report only reconstruction R^2 .

As in the digits setting, TPE reconstruction generalizes to unseen filler-role combinations (Table J.2). The average R^2 on the SVO generalization split is 0.9514, compared to 0.9727 on the in-distribution test split. This result indicates that the TPEs capture filler-role structures within the neural network activations that transfer to systematic substitutions.

Table 11: SVO sentence-embedding filler-role holdout reconstruction results.

Model	Test R^2	Generalization R^2
ModernBERT	0.9679	0.9589
EmbeddingGemma	0.9883	0.9602
Qwen3-Embedding-8B	0.9619	0.9350